

kubeadm 클러스터 + ArgoCD 구성 가이드

VM 3대에서 동작하는 클러스터까지 — 2026년 8월 정리

Proxmox에 VM 3대(마스터 1, 워커 2)를 준비한 다음부터의 기록이다. VM을 어떻게 만들었는지는 [proxmox-vm-setup-guide.pdf](#) 에 따로 있고, 여기서는 그 안에서 실제로 동작하는 쿠버네티스 클러스터를 어떻게 세웠는지만 다룬다.

k8s-master	192.168.1.101, 2vCPU / 3GB
k8s-worker1	192.168.1.102, 2vCPU / 4GB
k8s-worker2	192.168.1.103, 2vCPU / 4GB
계정	ubuntu
k8s 버전	1.35 (kubeadm)
CNI	Calico, pod-network-cidr 10.244.0.0/16

세 노드 공통 사전 작업

마스터, 워커1, 워커2 세 곳에 전부 동일하게 실행한다. kubelet이 정상적으로 뜨려면 스왑을 끄고, 컨테이너 네트워킹에 필요한 커널 설정을 넣고, 컨테이너 런타임과 kubeadm 도구 자체를 설치하는 네 단계를 거친다.

스왑 비활성화

kubelet은 스왑이 켜져 있으면 시작 자체를 거부한다. 메모리 관리를 예측 가능한 범위에 서만 하겠다는 설계 때문이다.

```
sudo swapoff -a
sudo sed -i '/swap/s/^/#/' /etc/fstab
```

두 번째 줄은 재부팅 후에도 스왑이 다시 켜지지 않도록 fstab에서 해당 줄을 주석 처리 해두는 것이다.

커널 모듈 로드

컨테이너 네트워킹에 필요한 두 모듈을 활성화한다.

```
cat <<EOF | sudo tee /etc/modules-load.d/k8s.conf
overlay
br_netfilter
EOF
sudo modprobe overlay
sudo modprobe br_netfilter
```

`overlay` 는 containerd가 이미지 레이어를 쌓을 때 쓰는 파일시스템이고, `br_netfilter` 는 브리지 네트워크를 지나는 패킷도 iptables 규칙의 대상이 되게 해준다. 이게 없으면 파드끼리 네트워킹이 제대로 안 된다.

네트워크 sysctl 설정

```
cat <<EOF | sudo tee /etc/sysctl.d/k8s.conf
net.bridge.bridge-nf-call-iptables = 1
net.bridge.bridge-nf-call-ip6tables = 1
net.ipv4.ip_forward = 1
EOF
sudo sysctl --system
```

앞의 두 줄이 방금 로드한 `br_netfilter` 를 실제로 활성화하는 스위치이고, `ip_forward` 는 이 노드가 받은 패킷을 다른 목적지로 전달할 수 있게 해준다. 파드 간 통신에 필요한 설정이다.

containerd 설치

```
sudo apt update && sudo apt install -y containerd
sudo mkdir -p /etc/containerd
containerd config default | sudo tee /etc/containerd/config.toml
sudo sed -i 's/SystemdCgroup = false/SystemdCgroup = true/' /etc/containerd/config.toml
sudo systemctl restart containerd
```

마지막에서 두 번째 줄이 핵심이다. cgroup 드라이버를 systemd로 맞춰야 Ubuntu(systemd 기반)와 kubelet의 리소스 관리 방식이 일치한다. 안 맞추면 리소스 제한이 꼬이면서 노드가 불안정해지는 경우가 흔하다.

kubeadm / kubelet / kubectl 설치

v1.35 기준이다.

```
sudo apt install -y apt-transport-https ca-certificates curl gpg
sudo mkdir -p /etc/apt/keyrings
curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.35/deb/Release.key | sudo gpg --dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg
echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] https://pkgs.k8s.io/core:/stable:/v1.35/deb/ /' | sudo tee /etc/apt/sources.list.d/kubernetes.list
sudo apt update
sudo apt install -y kubelet kubeadm kubectl
sudo apt-mark hold kubelet kubeadm kubectl
```

마지막 줄(`apt-mark hold`)은 이후 `apt upgrade` 를 돌려도 이 세 패키지 버전이 실수로 바뀌지 않게 고정해두는 것이다. k8s는 마이너 버전 하나만 바뀌어도 클러스터가 깨질 수 있어서 꼭 필요한 습관이다.

마스터에서만: 클러스터 초기화

```
sudo kubeadm init --pod-network-cidr=10.244.0.0/16 --apiserver-advertise-address=192.168.1.101

mkdir -p $HOME/.kube
```

```
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

`pod-network-cidr` 은 파드에게 할당되는 내부 IP 대역이다. 집 LAN 대역(192.168.1.0/24) 과 겹치지 않는 값을 써야 하는데, 이걸 놓치고 Calico 기본값을 그대로 썼다가 나중에 다시 잡아야 했다. 명령어 실행이 끝나면 출력 맨 아래에 `kubeadm join` 명령어(토큰 포함) 가 나오는데, 워커를 붙이려면 이 값을 복사해둬야 한다.

마스터에서만: Calico 설치

```
curl https://raw.githubusercontent.com/projectcalico/calico/v3.28.0/manifests/tigera-operator.yaml -O
kubectl create -f tigera-operator.yaml
curl https://raw.githubusercontent.com/projectcalico/calico/v3.28.0/manifests/custom-resources.yaml -O
```

받은 `custom-resources.yaml` 을 열어서 `cidr: 192.168.0.0/16` 으로 되어 있는 기본값을 위에서 지정한 `10.244.0.0/16` 으로 바꾼 다음 적용한다.

```
kubectl create -f custom-resources.yaml
```

워커에서: 클러스터 합류

워커1, 워커2 각각에서 실행한다.

```
sudo kubeadm join 192.168.1.101:6443 \
  --token <마스터에서 복사한 토큰> \
  --discovery-token-ca-cert-hash sha256:<해시>
```

토큰이 "이 워커가 이 클러스터에 합류해도 되는 신뢰된 요청인지"를 증명하는 역할을 한다.

검증

```
watch kubectl get pods -n calico-system
kubectl get nodes -o wide
```

calico-system 네임스페이스의 파드들이 전부 Running이 되고 나면(1~2분 정도 걸린다), 노드 3대가 전부 Ready로 뜨는지 확인한다. 워커는 kubeadm join 만으로는 ROLES 칸이 <none>으로 뜨는데 정상이다 — kubeadm이 워커에 별도 role 라벨을 자동으로 붙여주지 않을 뿐 기능상 문제는 없다. 보기 좋게 하고 싶으면 직접 라벨을 붙이면 된다.

```
kubectl label node k8s-worker1 node-role.kubernetes.io/worker=worker
kubectl label node k8s-worker2 node-role.kubernetes.io/worker=worker
```

kubectl 자동완성 + 단축 명령

```
sudo apt install -y bash-completion
echo 'source <(kubectl completion bash)' >> ~/.bashrc
echo 'alias k=kubectl' >> ~/.bashrc
echo 'complete -o default -F __start_kubectl k' >> ~/.bashrc
source ~/.bashrc
```

.bashrc에 추가만 하고 source를 안 하면 그 터미널 세션에는 반영이 안 된다 — 새 SSH 세션을 열거나 마지막 줄을 직접 실행해야 자동완성과 k 단축 명령이 살아난다.

Helm + ArgoCD 설치

Helm 설치

```
curl -fsSL -o get_helm.sh https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3
chmod 700 get_helm.sh
./get_helm.sh
```

ArgoCD 설치

```
helm repo add argo https://argoproj.github.io/argo-helm
helm repo update
kubectl create namespace argocd
helm install argocd argo/argo-cd -n argocd
```

초기 비밀번호 확인

```
kubectl -n argocd get secret argocd-initial-admin-secret -o jsonpath="{.data.password}" | base64 -d
```

계정은 `admin`, 비밀번호는 위 명령어 출력값이다. 이 시크릿은 1회용이라 로그인 후 비밀번호를 바꾸면 자동으로 삭제된다.

외부 접근용 NodePort 전환

처음엔 `kubectl port-forward` 로 접속했는데, 이건 그 명령어를 실행한 터미널이 살아있는 동안만 유지되는 임시 연결이라 세션이 끊기면 같이 끊긴다. 상시 접근이 필요해서 NodePort로 바꿨다.

```
cat > argocd-values.yaml << 'EOF'
server:
  service:
    type: NodePort
    nodePortHttps: 30443
EOF

helm upgrade argocd argo/argo-cd -n argocd -f argocd-values.yaml
```

`-f` 로 넘기는 값은 차트를 바꾸는 게 아니라, 차트 개발자가 이미 만들어둔 파라미터 (`server.service.type` 같은 것)에 값을 채워넣는 것이다. 어떤 값이 있는지는 `helm show values argo/argo-cd` 로 전체 기본값을 확인하거나, `helm pull argo/argo-cd --untar` 후 `grep` 으로 실제 템플릿에서 그 값이 쓰이는 위치를 찾아서 확인할 수 있다.

```
kubectl get svc argocd-server -n argocd
```

PORT(S) 항목이 443:30443/TCP 로 나오면 성공이다. 이후

<https://192.168.1.101:30443> 으로 바로 접속된다.

겪었던 실수 하나: `helm upgrade` 는 릴리스 이름과 차트 경로 두 개를 순서대로 받는데, 릴리스 이름(`argocd`)을 빼먹으면 "requires 2 arguments" 에러가 난다. 설치할 때 지은 릴리스 이름을 항상 맨 앞에 붙여야 한다.